2022

101Alpha 因子的快速提取

应用专题-数据提取专题 07

深圳天软科技

本文档所有内容均属于深圳市天软科技开发有限公司的商业机密，深圳市天软科技开发有限公司亦保留上述文档中阐述的包括而不限于技术、方法、思想的所有权。此文档仅仅只许可因本文档所必需传播的范围内使用（仅限于内部），本文档的获得者不得将本文档散发给不相关的其他机构和个人。

深圳天软科技 | 深圳市新闻路 1 号中电信息大厦 301-5

1 更新日志

1.1 更新日志

更新日志	更新说明
2018-11-26	1、创建文档
2018-11-30	2、发布 3、目前的入库的因子编号有 Alpha003,Alpha004,Alpha005,Alpha006,Alpha008, Alpha011,Alpha012,Alpha013,Alpha014,Alpha015, Alpha018,Alpha019,Alpha021,Alpha022,Alpha023 Alpha024,Alpha025,Alpha026,Alpha028,Alpha030 4、入库开始日期 2018-01-01 日 5、后续的更新以及因子、日期的扩大范围都在此更新日记发布
2022-03-10	废弃入库说明，废弃 Get101alphaByID 的变更说明

1.2 摘要

1) 给出了 101Alpaha 因子封装的标准以及实时调用方法

2) 给出了快速取到 101Alpaha 因子的方法 **Get101alphaByID** 函数，此函数从数据仓库取到因子值

3) 第五章节才是重点，提供了从数据仓库取数，而非实时计算因子值。

4) 目前的因子入库还不支持把所有的因子入库，具体哪些因子查看更新日记列表。

2 关于 101Alpaha 因子

2015 年 9 月，Zura Kakushadze 在他的论文《101 Formulaic Alphas》中公布了对冲基金 WorldQuant(世坤投资) 101 个阿尔法表达式，其中 80%仍然在实盘中使用。我们把这些公式编写成了天软函数，供大家使用，我们也称之为 101Alpaha 因子。论文地址：https://arxiv.org/pdf/1601.00991.pdf。

本文主要介绍两部分：

1) 101Alpaha 因子公式常使用到的算法，如全市场排名 rank(x)等

2) 如何从天软快速取到 101Alpaha 因子值。

至于 101Alpaha 因子中哪些因子具有显著性或者说能否适合用在 A 股市场等问题，需要大家去探究，本文致力于大家在探究 101Alpaha 因子时提供一些中间环节的帮助。

3 101Alpaha 因子计算函数

3.1 函数开发标准

101Alpaha 因子函数在天软中的路径为：**公用函数\\扩展函数\\.Net 扩展函数\\技术分析**\\101Alpaha 文件夹下，所有因子 101Alpaha 的函数名称以 OHO_开头作为前缀。

101Alpaha 因子计算时，大部分的算法不仅会计算和其他股票对比的排名，也会用到个股自身历史数据，这就引发出两个问题：

1) 如果要进行全市场排名时，股票池的范围如何定义，是否要考虑剔除停牌等因素？

2) 取历史数据时，遇到停牌如何处理？

针对上述问题，我们给出了 101Alpaha 函数规则：

- 计算全市场排名时，用来排名的股票池范围需要用户设置，详情看范例 3.1.1；
- 对于停牌个股，停牌期间不予计算，一律返回 NAN 值；
- 对于正常交易的个股，使用到历史数据时，会考虑时间对齐，一律使用市场交易日序列；
- 对于正常交易的个股，使用市场交易日序列取历史数据，会遇到历史停牌的情况， 101Alpaha 因子开发的标准是，删掉停牌时的数据，若因删掉数据而计算不出最终的因子值时，一律返回 NAN 值，举个例子，取某股票前 10 日的数据算标准差，但前 9 日处于停牌阶段，属于当前才复牌交易的个股，这种情况只有 1 日数据计算不出标准差时，会返回 nan 值。

3.1.1 取数范例 1

```
StockArr:=GetBK("沪深 300");
```

```
SetSysParam(pn_StockArr(),StockArr);  //注意设置 pn_StockArr()
SetSysParam(PN_Cycle(),cy_day());
SetSysParam(PN_Rate(),1);
r:=array();
n:=0;
for nI:=0 to length(StockArr)-1 do
begin
    StockId:=StockArr[nI];
    echo StockId;
    SetSysParam(PN_Stock(),StockId);
    SetSysParam(PN_Date(),EndT);
    r[n]['代码']:=StockId;
    r[n]['名称']:=StockName(StockId);
    r[n]['Alph002']:=OHO_Alpha_002();
    r[n]['Alph003']:=OHO_Alpha_003();
    //其他的模型类似的调用方法，如： OHO_Alpha_xxx，其中 xxx 为对应的序号
    //r[n]['Alph00N']:=OHO_Alpha_xxx();
    n++;
end
return r;
```

3.2 常用算法

101Alpaha 因子中一些常用的算法与天软函数对照表。

序号	101 Formulaic Alphas	天软函数	备注
01	rank（x）	PercentRankOftheDay(Exp,CacheStr)	参见 3.1.1
02	delay(x,d)	DeltaOfNDay(Exp,N)	3.1.2
03	correlation(x,y,d)	CorrelOfNDay(Exp1,Exp2,N)	3.1.3
04	covariance(x,y,d)	CovarOfNDay(Exp1,Exp2,N)	3.1.4
05	scale(x,a)	ScaleOfSeries(Exp,a,CacheStr)	3.1.5
06	delta(x,d)	DeltaOfNDay(Exp,N)	3.1.6
07	signedpower(x,a)	SignedPower(Exp,N)	3.1.7
08	decay_linear(x,d)	DecayAvgOfNDay(Exp,N)	3.1.8
09	indneutralize(x,g)	IndNeutralize(Exp,ByGroup,CacheStr)	3.1.9
10	ts_min(x,d)	LowestOfNDay(Exp,N)	3.1.10

11	ts_max(x,d)	HighestOfNDay(Exp,N)	3.1.11
12	ts_argmin(x,d)	LLVBarsOfNDay(Exp,N)	3.1.12
13	ts_argmax(x,d)	HHVBarsOfNDay(Exp,N)	3.1.13
14	ts_rank(x,d)	PercentRankOfNDay(Exp,N)	3.1.14
15	sum(x,d)	SumOfNDay(Exp,N)	3.1.15
16	product(x,d)	ProductOfNDay(Exp,N)	3.16
17	stddev(x,d)	StdOfNDay(Exp,N)	3.17

3.2.1 Rank(x)

cross-sectional rank，股票横截面数据排名，x 表示一组股票的截面数据，按向量 x 的元素大小升序排序，序号从 0 开始，如[0,1,...,n]，然后输出排名所占总位数的百分比，即将排名归一到[0,1]区间。例如：输入原始 x 向量，x=[6,4,6,2,9,9]，升序排名数组 r=[2,1,2,0,4,4]，再将 r/4，归一化后输出数组[0.5,0.25,0.5,0,1,1]。

在天软对应的函数为 PercentRankOftheDay(Exp,CacheStr)，函数说明参照下节。

3.2.1.1 PercentRankOftheDay(Exp,CacheStr)

- 定义： PercentRankOftheDay(Exp,CacheStr)
- 说明：当日分位排名
- 参数：Exp 表达式类型 ，指要进行排名的表达式

  CacheStr 字符串类型，缓存作用，将会被当做唯一标识缓存数据，输入必须是唯一串，详情看 6.1 中对该参数的说明
- 返回：排名

3.2.2 delay(x,d)

x 变量 d 天前的值，特别 d 天指的是市场的 d 天，非个股的 d 天，因为存在个股停牌的原因，如计算时遇到 d 天前为停牌期间，则返回 nan 无效。

在天软对应的函数为 DeltaOfNDay(Exp,N)，函数说明参照下节

3.2.2.1 DeltaOfNDay(Exp,N)

- 定义：DeltaOfNDay(Exp,N)
- 说明：返回 N 日前的值
- 参数：Exp 表达式类型

  N 整数型，N 日前
- 返回：N 日前的值

3.2.3 correlation(x,y,d)

time-serial correlation of x and y for the past d days ，x 和 y 两个变量过去 d 天时间序列的相关系数。在天软对应的函数为 CorrelOfNDay(Exp1,Exp2,N)，函数说明参照下节

3.2.3.1 CorrelOfNDay(Exp1,Exp2,N)

- 定义：CorrelOfNDay(Exp1,Exp2,N)
- 说明：返回 Exp1,Exp2 N 日的相关系数，考虑时间对齐
- 参数：Exp1 表达式类型

  Exp2 表达式类型

  N 整数型，N 日
- 返回：相关系数

3.2.4 covariance(x,y,d)

time-serial covariance of x and y for the past d days ，x 和 y 两个变量过去 d 天时间序列的协方差，在天软对应的函数为 CorrelOfNDay(Exp1,Exp2,N)，函数说明参照下节

3.2.4.1 CovarOfNDay(Exp1,Exp2,N)

- 定义：CorrelOfNDay(Exp1,Exp2,N)
- 说明：过去 N 日协方差，其中 N 日表示市场交易日
- 参数：Exp1 表达式类型

  Exp2 表达式类型

  N 整数型，N 日
- 返回：协方差

3.2.5 scale(x,a)

rescaled x such that sum(abs(x))=a (the default is a=1) ，将 x 中的值标准化，使 x 的绝对值的和为 a，默认 a=1，在天软对应的函数为 ScaleOfSeries(Exp,a,CacheStr)，函数说明参照下节

3.2.5.1 ScaleOfSeries(Exp,a,CacheStr)

- 定义：ScaleOfSeries(Exp,a,CacheStr)
- 说明：将 x 中的非 NAN 值进行标准化，使 x 的绝对值的和为 a，默认 a=1
- 参数：Exp 表达式类型

  A 浮点型

CacheStr　字符型，缓存串，详情查看 6.1

- 返回：标准化之后结果

3.2.6　delta(x,d)

today’s value of x minus the value of x d days ago，指定 endt 的 x 值减去 d 天之前的 x 值，在天软对应的函数为 DeltaOfNDay(Exp,N)，函数说明参照下节

3.2.6.1　DeltaOfNDay(Exp,N)

- 定义：DeltaOfNDay(Exp,N)
- 说明：endt 的 x 值减去 d 天之前的 x 值
- 参数：Exp 表达式类型

  N　整型　N 日
- 返回：两者的差值

3.2.7　signedpower(x,a)

x^a　or　sign(x)*(abs(x)^a)，保持向量 x 的正负性，求 x 绝对值的 a 次方，在天软对应的函数为 signedpower(x,a)，函数说明参照下节

3.2.7.1　SignedPower(Exp,N)

- 定义：SignedPower(Exp,N)
- 说明：符号次方
- 参数：Exp 表达式类型

  N　整型　N 日
- 返回：符号次方

3.2.8　decay_linear(x,d)

weighted moving average over the past d days with linearly decaying weights d,d-1,…,1 (rescaled up to 1)，过去 d 天的加权移动平均,权值线性衰减 d,d-1,...,1(和为 1)，如：

decay_linear(x,d)=(x[endt-1]*d+x[endt-2]*(d-1)+...+x[endt-d]*1)/(d+(d-1)+...+1)

在天软对应的函数为 DecayAvgOfNDay(Exp,N)，函数说明参照下节

3.2.8.1　DecayAvgOfNDay(Exp,N)

- 定义：DecayAvgOfNDay(Exp,N)

- 说明：N 日线性衰减加权平均
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.9 indneutralize(x,g)

x cross-sectionally neutralized against groups g (subindustries, industries, sectors, etc.),i.e., x is cross-sectionally demeaned within each group g

中性化操作，将 x 按照 g 分类，对每个类别按照 alpha = alpha － mean(alpha)进行中性化处理，之后 x 的均值为 0。例如，当 g=“market”，即对整个市场的 x 值进行中性化，x=x-mean(x)；当 g=“industry”，则先对整个市场按照“industry”进行分类，再对每个类别分别进行中性化处理，如第 i 个股票属于“industry-1”，则 x(i)=x(i) - mean(x in industry-1).

在天软对应的函数为 IndNeutralize(Exp,ByGroup,CacheStr)，函数说明参照下节

3.2.9.1 IndNeutralize(Exp,ByGroup,CacheStr)

- 定义：IndNeutralize(Exp,ByGroup,CacheStr)
- 说明：中性化
- 参数：Exp 表达式类型

  ByGroup 整数型 0 表示整个市场

  CacheStr 字符串类型，缓存作用，将会被当做唯一标识缓存数据，输入必须是唯一串，详情看 6.1 中对该参数的说明
- 返回：实数

3.2.10 ts_min(x,d)

time-series min over the past d days，x 在过去 d 天内最小的值，d 日内表示市场的 d 日，在天软对应的函数为 LowestOfNDay(Exp,N)，函数说明参照下节

3.2.10.1 LowestOfNDay(Exp,N)

- 定义：LowestOfNDay (Exp,N)
- 说明：N 日最小值
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.11 ts_max (x,d)

time-series max over the past d days，x 在过去 d 天内最大的值。d 日内表示市场的 d 日，在天软对应的函数为 HighestOfNDay(Exp,N)，函数说明参照下节

3.2.11.1 HighestOfNDay(Exp,N)

- 定义：HighestOfNDay(Exp,N)
- 说明：N 日最大值
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.12 ts_argmin(x,d)

which day ts_min(x,d) occurred on ，ts_min(x,d)发生在过去 d 天中的第几天，例如 x 在过去 d 天的值分别为（10,8,7,4,6）（按照时间升序排列），对应的索引号为 L=[5,4,3,2,1]，其中，索引号为 5 代表过去第 5 天，索引号为 1 代表过去第 1 天。因为 x 中最小的元素为 4，则索引号为 2，即过去第 2 天。在天软对应的函数为 HighestOfNDay(Exp,N)，函数说明参照下节

3.2.12.1 LLVBarsOfNDay(Exp,N);

- 定义：LLVBarsOfNDay (Exp,N)
- 说明：N 日最低点到当前的周期数
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.13 ts_argmax(x,d)

ts_max(x,d)发生在过去 d 天中的第几天，如上例，因为 x 中最大的元素为 10，则索引号为 5，即过去第 5 天。在天软对应的函数为 HHVBarsOfNDay (Exp,N)，函数说明参照下节

3.2.13.1 HHVBarsOfNDay(Exp,N)

- 定义：HHVBarsOfNDay (Exp,N)
- 说明：N 日最高点到当前的周期数
- 参数：Exp 表达式类型

N 整型 N 日

- 返回：实数

3.2.14 ts_rank(x,d)

对 x 过去 d 天的值（按照时间升序排列），进行 rank，输出最后一天的排名，即 rank[d-1]，在天软对应的函数为 PercentRankOfNDay(Exp,N)，函数说明参照下节

3.2.14.1 PercentRankOfNDay(Exp,N)

- 定义：PercentRankOfNDay (Exp,N)
- 说明：N 日最高点到当前的周期数
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.15 sum(x,d)

time-series sum over the past d days，对 x 过去 d 天的值累计求和，在天软对应的函数为 SumOfNDay(Exp,N)，函数说明参照下节

3.2.15.1 SumOfNDay(Exp,N)

- 定义：SumOfNDay (Exp,N)
- 说明：N 日和
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

3.2.16 product(x,d)

time-series product over the past d days，对 x 过去 d 天的值累计相乘，d 日指市场的 d 日，在天软对应的函数为 ProductOfNDay (Exp,N)，函数说明参照下节

3.2.16.1 ProductOfNDay(Exp,N)

- 定义：ProductOfNDay (Exp,N)
- 说明：N 日累乘
- 参数：Exp 表达式类型

N 整型 N 日

- 返回：实数

3.2.17 stddev(x,d)

moving time series standard deviation over the past d days , d 日指市场的 d 日,对 x 过去 d 天的值求标准差,在天软对应的函数为 ProductOfNDay (Exp,N)，函数说明参照下节

3.2.17.1 StdOfNDay(Exp,N)

- 定义：ProductOfNDay (Exp,N)
- 说明：N 日标准差
- 参数：Exp 表达式类型

  N 整型 N 日
- 返回：实数

4 计算函数的缓存机制

101aphla 因子中，很多算法比较复杂，而且还用到了全市场的数据，举个简单的例子，计算一只股票的因子值的排名，已经算好了，如果计算第二只股票时，又要计算全市场排名，是一件开销很大的事情，因此 101aphla 因子函数封装时使用的了缓存机制，启动缓存机制后，避免了重复计算的命运，加快了速度。

比较常见的就是算法函数中的最后一个参数总是带有 CacheStr，这个参数起到一个缓存串的作用，而且是唯一的串，一般这个参数能作为唯一串应该是由因子名称+日期+算法合成，如果是单个因子中需要缓存多个中间变量，则需要加入序号即可。举个例子如下图：

```
Function OHO_Alpha_050();
Begin
{
  Alpha#50:(-1*ts_max(rank(correlation(rank(volume),rank(vwap),5)),5))
}
   s:=functionname(); //缓存串
   s1:=s+"01";
   s2:=s+"02";
   s3:=s+"03";
   if not IsStockTradeDay(sp_time(1)) then return nan
   else
   begin
      index1:= @PercentRankOftheDay(@stockpjcj3(),s1);
      index2:= @PercentRankOftheDay(@vol(),s2);
      index3:= @CorrelOfNDay(index1,index2,5);
      return -HighestOfNDay(@PercentRankOftheDay(index3,s3),5);
   end;

End;
```

OHO_Alpha_050()的封住就缓存了三个全市场中间变量，因此同一个因子缓存多个中间变量时需要区分，避免缓存混乱。即使是带这里，从时间维度的缓存还是没有解决，因此打开上图中椭圆圈的函数 PercentRankOftheDay 的函数体，如下图，就会发现，CacheStr 作为参数传进来并不能作为唯一的串，还得需要加入日期+以及算法函数名称，如下图中红色长方形才是唯一的缓存串。其他算法的缓存机制也是类似。而在计算结束后会释放掉所有缓存。

```
1  Function PercentRankOftheDay(Exp, CacheStr);
2  Begin
3  {
4    股票横截面数据排名，x表示一组股票的截面数据，按向量x的元素大小升序排序，序号从0开始，
5    如[0,1,...,n]，然后输出排名所占总位数的百分比，即将排名归一到[0,1]区间。
6    Nan值不参与排名，输出Nan。
7    例如：输入原始x向量，x=[6,4,6,2,9,9]，升序排名数组r=[2,1,2,0,4,4]，再将r/(length(x)-1)
8          归一化后输出数组[0.4,0.2,0.4,0,0.8,0.8]。
9          若含有nan，x=[1,nan,2,5]，输出[0,nan,0.5,1]
10
11   加一个参数CacheStr解决缓存-2018-08-31
12
13 }
14
15   t:=static GetDataSub(Exp) name functionname()+CacheStr+datetostr(sp_time());
16   StockId:=DefaultStockID();
```

5 如何快速取 101Alpaha 因子

《2019-07-29-应用专题-数据提取专题 08：101Alpha 因子升级及效率优化说明》中 **4、101 因子计算模型汇总中的高级接口。**

6 附录

6.1 101ALpaha 因子表达式

Alpha#1:(rank(Ts_ArgMax(SignedPower(((returns<0)?stddev(returns,20):close),2.),5))-0.5)

Alpha#2:(-1*correlation(rank(delta(log(volume),2)),rank(((close-open)/open)),6))

Alpha#3:(-1*correlation(rank(open),rank(volume),10))

Alpha#4:(-1*Ts_Rank(rank(low),9))

Alpha#5:(rank((open-(sum(vwap,10)/10)))*(-1*abs(rank((close-vwap)))))

Alpha#6:(-1*correlation(open,volume,10))

Alpha#7:((adv20<volume)?((-1*ts_rank(abs(delta(close,7)),60))*sign(delta(close,7))):(-1*1))

Alpha#8:(-1*rank(((sum(open,5)*sum(returns,5))-
((sum(open,5)*sum(returns,5)),10))))

Alpha#9:((0<ts_min(delta(close,1),5))?delta(close,1):((ts_max(delta(close,1),5)<0)?delta(close,1):(-1*delta(close,1))))

Alpha#10:rank(((0<ts_min(delta(close,1),4))?delta(close,1):((ts_max(delta(close,1),4)<0)?delta(close,1):(-1*delta

(close,1)))))

Alpha#11:((rank(ts_max((vwap-close),3))+rank(ts_min((vwap-close),3)))*rank(delta(volume,3)))

Alpha#12:(sign(delta(volume,1))*(-1*delta(close,1)))

Alpha#13:(-1*rank(covariance(rank(close),rank(volume),5)))

Alpha#14:((-1*rank(delta(returns,3)))*correlation(open,volume,10))

Alpha#15:(-1*sum(rank(correlation(rank(high),rank(volume),3)),3))

Alpha#16:(-1*rank(covariance(rank(high),rank(volume),5)))

Alpha#17:(((-1*rank(ts_rank(close,10)))*rank(delta(delta(close,1),1)))*rank(ts_rank((volume/adv20),5)))

Alpha#18:(-1*rank(((stddev(abs((close-open)),5)+(close-open))+correlation(close,open,10))))

Alpha#19:((-1*sign(((close-delay(close,7))+delta(close,7))))*(1+rank((1+sum(returns,250)))))

Alpha#20:(((-1*rank((open-delay(high,1))))*rank((open-delay(close,1))))*rank((open-delay(low,1))))

Alpha#21:((((sum(close,8)/8)+stddev(close,8))<(sum(close,2)/2))?(-1*1):(((sum(close,2)/2)<((sum(close,8)/8)-std
dev(close,8)))?1:(((1<(volume/adv20))||((volume/adv20)==1))?1:(-1*1))))

Alpha#22:(-1*(delta(correlation(high,volume,5),5)*rank(stddev(close,20))))

Alpha#23:(((sum(high,20)/20)<high)?(-1*delta(high,2)):0)

Alpha#24:((((delta((sum(close,100)/100),100)/delay(close,100))<0.05)||((delta((sum(close,100)/100,100)/delay(
close,100))==0.05))?(-1*(close-ts_min(close,100))):(-1*delta(close,3)))

Alpha#25:rank(((((-1*returns)*adv20)*vwap)*(high-close)))

Alpha#26:(-1*ts_max(correlation(ts_rank(volume,5),ts_rank(high,5),5),3))

Alpha#27:((0.5<rank((sum(correlation(rank(volume),rank(vwap),6),2)/2.0)))?(-1*1):1)

Alpha#28:scale(((correlation(adv20,low,5)+((high+low)/2))-close))

Alpha#29:(min(product(rank(rank(scale(log(sum(ts_min(rank(rank((-1*rank(delta((close-1),5))))),2),1))))),1),5)+
ts_rank(delay((-1*returns),6),5))

Alpha#30:(((1.0-rank(((sign((close-delay(close,1)))+sign((delay(close,1)-delay(close,2))))+sign((delay(close,2)-
delay(close,3))))))*sum(volume,5))/sum(volume,20))

Alpha#31:((rank(rank(rank(decay_linear((-1*rank(rank(delta(close,10))))),10))))+rank((-1*delta(close,3))))+sign(
scale(correlation(adv20,low,12))))

Alpha#32:(scale(((sum(close,7)/7)-close))+(20*scale(correlation(vwap,delay(close,5),230))))

Alpha#33:rank((-1*((1-(open/close))^1)))

Alpha#34:rank(((1-rank((stddev(returns,2)/stddev(returns,5))))+(1-rank(delta(close,1)))))

Alpha#35:((Ts_Rank(volume,32)*(1-Ts_Rank(((close+high)-low),16)))*(1-Ts_Rank(returns,32)))

Alpha#36:(((((2.21*rank(correlation((close-open),delay(volume,1),15)))+(0.7*rank((open-close))))+(0.73*rank(
Ts_Rank(delay((-1*returns),6),5))))+rank(abs(correlation(vwap,adv20,6))))+(0.6*rank((((sum(close,200
)/200)-open)*(close-open)))))

Alpha#37:(rank(correlation(delay((open-close),1),close,200))+rank((open-close)))

Alpha#38:((-1*rank(Ts_Rank(close,10)))*rank((close/open)))

Alpha#39:((-1*rank((delta(close,7)*(1-rank(decay_linear((volume/adv20),9))))))*(1+rank(sum(returns,250))))

Alpha#40:((-1*rank(stddev(high,10)))*correlation(high,volume,10))

Alpha#41:(((high*low)^0.5)-vwap)

Alpha#42:(rank((vwap-close))/rank((vwap+close)))

Alpha#43:(ts_rank((volume/adv20),20)*ts_rank((-1*delta(close,7)),8))

Alpha#44:(-1*correlation(high,rank(volume),5))

Alpha#45:(-1*((rank((sum(delay(close,5),20)/20))*correlation(close,volume,2))*rank(correlation(sum(close,5),sum(close,20),2))))

Alpha#46:((0.25<(((delay(close,20)-delay(close,10))/10)-((delay(close,10)-close)/10)))?(-1*1):(((((delay(close,20)-delay(close,10))/10)-((delay(close,10)-close)/10))<0)?1:((-1*1)*(close-delay(close,1)))))

Alpha#47:((((rank((1/close))*volume)/adv20)*((high*rank((high-close)))/(sum(high,5)/5)))-rank((vwap-delay(vwap,5))))

Alpha#48:(indneutralize(((correlation(delta(close,1),delta(delay(close,1),1),250)*delta(close,1))/close),IndClass.subindustry)/sum(((delta(close,1)/delay(close,1))^2),250))

Alpha#49:(((((delay(close,20)-delay(close,10))/10)-((delay(close,10)-close)/10))<(-1*0.1))?1:((-1*1)*(close-delay(close,1))))

Alpha#50:(-1*ts_max(rank(correlation(rank(volume),rank(vwap),5)),5))

Alpha#51:(((((delay(close,20)-delay(close,10))/10)-((delay(close,10)-close)/10))<(-1*0.05))?1:((-1*1)*(close-delay(close,1))))

Alpha#52:((((-1*ts_min(low,5))+delay(ts_min(low,5),5))*rank(((sum(returns,240)-sum(returns,20))/220)))*ts_rank(volume,5))

Alpha#53:(-1*delta((((close-low)-(high-close))/(close-low)),9))

Alpha#54:((-1*((low-close)*(open^5)))/((low-high)*(close^5)))

Alpha#55:(-1*correlation(rank(((close-ts_min(low,12))/(ts_max(high,12)-ts_min(low,12)))),rank(volume),6))

Alpha#56:(0-(1*(rank((sum(returns,10)/sum(sum(returns,2),3)))*rank((returns*cap)))))

Alpha#57:(0-(1*((close-vwap)/decay_linear(rank(ts_argmax(close,30)),2))))

Alpha#58:(-1*Ts_Rank(decay_linear(correlation(IndNeutralize(vwap,IndClass.sector),volume,3.92795),7.89291),5.50322))

Alpha#59:(-1*Ts_Rank(decay_linear(correlation(IndNeutralize(((vwap*0.728317)+(vwap*(1-0.728317))),IndClass.industry),volume,4.25197),16.2289),8.19648))

Alpha#60:(0-(1*((2*scale(rank(((((close-low)-(high-close))/(high-low))*volume))))-scale(rank(ts_argmax(close,10))))))

Alpha#61:(rank((vwap-ts_min(vwap,16.1219)))<rank(correlation(vwap,adv180,17.9282)))

Alpha#62:((rank(correlation(vwap,sum(adv20,22.4101),9.91009))<rank(((rank(open)+rank(open))<(rank(((high+

low)/2))+rank(high)))))*-1)

Alpha#63:((rank(decay_linear(delta(IndNeutralize(close,IndClass.industry),2.25164),8.22237))-rank(decay_linear(correlation(((vwap*0.318108)+(open*(1-0.318108))),sum(adv180,37.2467),13.557),12.2883)))*-1)

Alpha#64:((rank(correlation(sum(((open*0.178404)+(low*(1-0.178404))),12.7054),sum(adv120,12.7054),16.6208))<rank(delta(((((high+low)/2)*0.178404)+(vwap*(1-0.178404))),3.69741)))*-1)

Alpha#65:((rank(correlation(((open*0.00817205)+(vwap*(1-0.00817205))),sum(adv60,8.6911),6.40374))<rank((open-ts_min(open,13.635))))*-1)

Alpha#66:((rank(decay_linear(delta(vwap,3.51013),7.23052))+Ts_Rank(decay_linear(((((low*0.96633)+(low*(1-0.96633)))-vwap)/(open-((high+low)/2))),11.4157),6.72611))*-1)

Alpha#67:((rank((high-ts_min(high,2.14593)))^rank(correlation(IndNeutralize(vwap,IndClass.sector),IndNeutralize(adv20,IndClass.subindustry),6.02936)))*-1)

Alpha#68:((Ts_Rank(correlation(rank(high),rank(adv15),8.91644),13.9333)<rank(delta(((close*0.518371)+(low*(1-0.518371))),1.06157)))*-1)

Alpha#69:((rank(ts_max(delta(IndNeutralize(vwap,IndClass.industry),2.72412),4.79344))^Ts_Rank(correlation(((close*0.490655)+(vwap*(1-0.490655))),adv20,4.92416),9.0615))*-1)

Alpha#70:((rank(delta(vwap,1.29456))^Ts_Rank(correlation(IndNeutralize(close,IndClass.industry),adv50,17.8256),17.9171))*-1)

Alpha#71:max(Ts_Rank(decay_linear(correlation(Ts_Rank(close,3.43976),Ts_Rank(adv180,12.0647),18.0175),4.20501),15.6948),Ts_Rank(decay_linear((rank(((low+open)-(vwap+vwap)))^2),16.4662),4.4388))

Alpha#72:(rank(decay_linear(correlation(((high+low)/2),adv40,8.93345),10.1519))/rank(decay_linear(correlation(Ts_Rank(vwap,3.72469),Ts_Rank(volume,18.5188),6.86671),2.95011)))

Alpha#73:(max(rank(decay_linear(delta(vwap,4.72775),2.91864)),Ts_Rank(decay_linear(((delta(((open*0.147155)+(low*(1-0.147155))),2.03608)/((open*0.147155)+(low*(1-0.147155))))*-1),3.33829),16.7411))*-1)

Alpha#74:((rank(correlation(close,sum(adv30,37.4843),15.1365))<rank(correlation(rank(((high*0.0261661)+(vwap*(1-0.0261661)))),rank(volume),11.4791)))*-1)

Alpha#75:(rank(correlation(vwap,volume,4.24304))<rank(correlation(rank(low),rank(adv50),12.4413)))

Alpha#76:(max(rank(decay_linear(delta(vwap,1.24383),11.8259)),Ts_Rank(decay_linear(Ts_Rank(correlation(IndNeutralize(low,IndClass.sector),adv81,8.14941),19.569),17.1543),19.383))*-1)

Alpha#77:min(rank(decay_linear(((((high+low)/2)+high)-(vwap+high)),20.0451)),rank(decay_linear(correlation(((high+low)/2),adv40,3.1614),5.64125)))

Alpha#78:(rank(correlation(sum(((low*0.352233)+(vwap*(1-0.352233))),19.7428),sum(adv40,19.7428),6.83313))^rank(correlation(rank(vwap),rank(volume),5.77492)))

Alpha#79:(rank(delta(IndNeutralize(((close*0.60733)+(open*(1-0.60733))),IndClass.sector),1.23438))<rank(correlation(Ts_Rank(vwap,3.60973),Ts_Rank(adv150,9.18637),14.6644)))

Alpha#80:((rank(Sign(delta(IndNeutralize(((open*0.868128)+(high*(1-0.868128))),IndClass.industry),4.04545)))
^Ts_Rank(correlation(high,adv10,5.11456),5.53756))*-1)

Alpha#81:((rank(Log(product(rank((rank(correlation(vwap,sum(adv10,49.6054),8.47743))^4)),14.9655)))<rank
(correlation(rank(vwap),rank(volume),5.07914)))*-1)

Alpha#82:(min(rank(decay_linear(delta(open,1.46063),14.8717)),Ts_Rank(decay_linear(correlation
(IndNeutralize(volume,IndClass.sector),((open*0.634196)+(open*(1-0.034196))),17.4842),6.92131),
13.4283))*-1)

Alpha#83:((rank(delay(((high-low)/(sum(close,5)/5)),2))*rank(rank(volume)))/(((high-low)/(sum(close,5)/5))/
(vwap-close)))

Alpha#84:SignedPower(Ts_Rank((vwap-ts_max(vwap,15.3217)),20.7127),delta(close,4.96796))

Alpha#85:(rank(correlation(((high*0.876703)+(close*(1-0.876703))),adv30,9.61331))^rank(correlation(Ts_Rank(
((high+low)/2),3.70596),Ts_Rank(volume,10.1595),7.11408)))

Alpha#86:((Ts_Rank(correlation(close,sum(adv20,14.7444),6.00049),20.4195)<rank(((open+close)-(vwap+open)
)))*-1)

Alpha#87:(max(rank(decay_linear(delta(((close*0.369701)+(vwap*(1-0.369701))),1.91233),2.65461)),Ts_Rank(
decay_linear(abs(correlation(IndNeutralize(adv81,IndClass.industry),close,13.4132)),4.89768),14.4535)
)*-1)

Alpha#88:min(rank(decay_linear(((rank(open)+rank(low))-(rank(high)+rank(close))),8.06882)),Ts_Rank(
decay_linear(correlation(Ts_Rank(close,8.44728),Ts_Rank(adv60,20.6966),8.01266),6.65053),2.61957))

Alpha#89:(Ts_Rank(decay_linear(correlation(((low*0.967285)+(low*(1-0.967285))),adv10,6.94279),5.51607),
3.79744)-Ts_Rank(decay_linear(delta(IndNeutralize(vwap,IndClass.industry),3.48158),10.1466),
15.3012))

Alpha#90:((rank((close-ts_max(close,4.66719)))^Ts_Rank(correlation(IndNeutralize(adv40,IndClass.subindustry)
,low,5.38375),3.21856))*-1)

Alpha#91:((Ts_Rank(decay_linear(decay_linear(correlation(IndNeutralize(close,IndClass.industry),volume,
9.74928),16.398),3.83219),4.8667)-rank(decay_linear(correlation(vwap,adv30,4.01303),2.6809)))*-1)

Alpha#92:min(Ts_Rank(decay_linear(((((high+low)/2)+close)<(low+open)),14.7221),18.8683),Ts_Rank(
decay_linear(correlation(rank(low),rank(adv30),7.58555),6.94024),6.80584))

Alpha#93:(Ts_Rank(decay_linear(correlation(IndNeutralize(vwap,IndClass.industry),adv81,17.4193),19.848),
7.54455)/rank(decay_linear(delta(((close*0.524434)+(vwap*(1-0.524434))),2.77377),16.2664)))

Alpha#94:((rank((vwap-ts_min(vwap,11.5783)))^Ts_Rank(correlation(Ts_Rank(vwap,19.6462),Ts_Rank(adv60,
4.02992),18.0926),2.70756))*-1)

Alpha#95:(rank((open-ts_min(open,12.4105)))<Ts_Rank((rank(correlation(sum(((high+low)/2),19.1351),sum(
adv40,19.1351),12.8742))^5),11.7584))

Alpha#96:(max(Ts_Rank(decay_linear(correlation(rank(vwap),rank(volume),3.83878),4.16783),8.38151),

Ts_Rank(decay_linear(Ts_ArgMax(correlation(Ts_Rank(close,7.45404),Ts_Rank(adv60,4.13242),3.65459),12.6556),14.0365),13.4143))*-1)

Alpha#97:((rank(decay_linear(delta(IndNeutralize(((low*0.721001)+(vwap*(1-0.721001))),IndClass.industry),3.3705),20.4523))-Ts_Rank(decay_linear(Ts_Rank(correlation(Ts_Rank(low,7.87871),Ts_Rank(adv60,17.255),4.97547),18.5925),15.7152),6.71659))*-1)

Alpha#98:(rank(decay_linear(correlation(vwap,sum(adv5,26.4719),4.58418),7.18088))-rank(decay_linear(Ts_Rank(Ts_ArgMin(correlation(rank(open),rank(adv15),20.8187),8.62571),6.95668),8.07206)))

Alpha#99:((rank(correlation(sum(((high+low)/2),19.8975),sum(adv60,19.8975),8.8136))<rank(correlation(low,volume,6.28259)))*-1)

Alpha#100:(0-(1*(((1.5*scale(indneutralize(indneutralize(rank(((((close-low)-(high-close))/(high-low))*vloume)),IndClass.subindustry),IndClass.subindustry)))-scale(indneutralize((correlation(close,rank(adv20),5)-rank(ts_argmin(close,30))),IndClass.subindustry)))*(volume/adv20))))

Alpha#101:((close-open)/((high-low)+.001))

7 FAQ

7.1 Q、Get101alphaByID 下架了，模型需要怎么替换?

原取数接口功能下架。101alpha 计算的时候需要指定股票池范围，因此用户需结合相关文档自己的需要，确定此参数。

所有用此模型取数接口都需按以下方式变更：

说明	原范例	新范例	注意事项
取单个因子	//获取万科 A 在 2022-02-23 的 101_001 因子 Endt := 20220223T; SetSysParam(PN_Stock(),"SZ000002"); SetSysParam(PN_Date(),Endt); return **QStock_101Alpha_001();**	//获取万科 A 的 101_001 因子，以当日 A 股为例 Endt := 20220223T; SetSysParam(PN_Stock(),"SZ000002"); SetSysParam(PN_Date(),Endt); **stks := GetAbkbyDate('A 股',Endt);** SetSysParam(pn_StockArr(),stks); **return OHO_Alpha_001();**	需要自己指定股票池范围

取多个因子	//取万科 10 天的 003、004 号因子
	setsysparam(pn_stock(),'SZ000002');
	setsysparam(pn_date(),today());
	SetSysParam(PN_Cycle(),cy_day());
	return nday(10,'date',sp_time(),'alpha3',Get101alphaByID(3),'alpha4',Get101alphaByID(4));
		//取万科 10 天的 003、004 号因子，以中证全指为例
		Endt := Today();
		Begt := MarketLastTradeDayN(10);
		setsysparam(pn_stock(),'SZ000002');
		setsysparam(pn_date(),Endt);
		SetSysParam(PN_Cycle(),cy_day());
		return **Stock_Alpha101_Cal(array(3,4),Begt,EndT,'SH000985');**
			需要自己指定股票池范围

备注：关于 Stock_Alpha101_Cal 的更多使用说明见**《2019-07-29-应用专题-数据提取专题 08：101Alpha 因子升级及效率优化说明》中 4、101 因子计算模型**